

Deep Learning Systems Analysis Report

Udacity Master's in AI, Deep Learning Systems Capstone

Belal Nwiran

Report Overview

This project classifies news articles into four topics (World, Sports, Business, and Sci/Tech) using the AG News dataset, which has 120,000 training and 7,600 test articles. I built a small Transformer encoder in PyTorch as a baseline and compared it with a version that differs only in dropout rate (0.1 versus 0.3). Both reached about 91% test accuracy (0.9058 and 0.9091), and the difference was not statistically significant. The main weakness of both models was confusion between Business and Sci/Tech articles.

GitHub repo: <https://github.com/bnwiran/deep-learning-systems-project>

Dataset and Task Description

The dataset is AG News, a collection of news articles labeled with one of four topics: World, Sports, Business, or Sci/Tech. Each record has a title, a short description, and a topic label. I downloaded the CSV files from Kaggle (the link is in the notebook) and did not modify them.

The training file has 120,000 articles and the test file has 7,600. Both are perfectly balanced, with 30,000 training and 1,900 test articles per class. I held out 10% of the training file as a validation set, using a stratified split with a fixed seed. This gives 108,000 training and 12,000 validation articles. The test set was used only once, for the final evaluation.

The task is text classification: predict the topic of an article from its title and description. The data fits this task because the texts are short (median 45 tokens, 99th percentile 108), every article has exactly one label, and no class dominates.

For preparation, I joined the title and description into one text, lowercased it, and split it into words and punctuation marks. The vocabulary was built from the training split only and has 30,000 entries: the most frequent training tokens plus padding and unknown tokens (the training split has 62,586 unique tokens). I chose this size because it covers 99.04% of training tokens, and going to 40,000 entries would add only about 0.5 points of coverage. About 1.30% of validation tokens map to the unknown token.

Every text was cut or padded to 128 tokens, because the 99.5th percentile of article length is 124 tokens; this affects fewer than 0.5% of training articles.

The data has no missing values and no duplicate rows. It contains encoded characters (for example "#39;" in place of an apostrophe), which I kept. About 20% of training articles (24,254) carry a news-agency tag such as Reuters or AP, and for some agencies the tags are unevenly spread across classes. I discuss this in the Limitations and Risks section.

Model Architecture and Design Decisions

The task needs a model that reads a whole article and gives one topic label. I used the encoder part of the Transformer design (Vaswani et al., 2017) and left out the decoder, because the model classifies text and does not generate it.

Each article is turned into 128 token IDs. An embedding layer maps each ID to a 128-dimensional vector, and these vectors are multiplied by the square root of the vector size, as in Vaswani et al. (2017). Self-attention does not know the order of words by itself, so a fixed sine and cosine position signal is added to the embeddings (Vaswani et al., 2017). The result passes through two encoder layers. Each layer has multi-head self-attention with four heads and a feed-forward network of size 256 with ReLU, with residual connections and layer normalization around both parts (Vaswani et al., 2017). A padding mask makes the model ignore padding positions. Table 1 summarizes the design.

Table 1*Baseline Model and Training Setup*

Component	Setting
Input	128 token IDs per article; vocabulary of 30,000 entries (including padding and unknown tokens)
Embedding	128 dimensions, multiplied by the square root of 128; padding ignored
Position signal	Fixed sine and cosine encoding added to the embeddings
Encoder	2 layers; 4 attention heads; feed-forward size 256 (ReLU); residual connections and layer normalization; dropout 0.1
Padding mask	Padding positions ignored in attention and in pooling
Pooling	Average of the output vectors of the real (non-padding) tokens
Output	Linear layer with 4 scores (logits); softmax is applied inside the loss
Size	4,105,476 parameters, about 3.84 million of them in the embedding table
Training	Cross-entropy loss; AdamW (learning rate 5e-4, weight decay 0.01); batch size 128; 10 epochs; gradient clipping at 1.0; seed 42; weights kept from the epoch with the best validation accuracy

The encoder gives one vector per token. I average the vectors of the real tokens to get one vector per article, and a linear layer turns it into four scores, one for each topic. I did not add a softmax layer, because the cross-entropy loss applies it internally. Averaging was my own design choice. It is simple and uses every token in the article.

The model has 4,105,476 parameters, and about 3.84 million of them (93.5%) are in the embedding table. It is much smaller than the base model in Vaswani et al. (2017), which has six layers and 512-dimensional vectors. I kept the model small so that training took about 15 seconds per epoch on one GPU and the experiment was easy to repeat. I did not tune the model size.

I trained with cross-entropy loss and the AdamW optimizer (learning rate 5e-4, weight decay 0.01), a batch size of 128, 10 epochs, and gradient clipping at 1.0. These values were a reasonable starting point and were not tuned. Ten epochs were enough for this setup: the best validation accuracy came at epoch 8 for the baseline and at epoch 9 for the dropout 0.3 model (Table 2). After each epoch the model was evaluated on the validation set, and the weights from the epoch with the best validation accuracy were kept. The test set was not used during training or model selection.

The baseline uses a dropout rate of 0.1 inside the encoder layers and before the output layer. This is the same value as the base setting reported by Vaswani et al. (2017). Here the dropout rate is the probability of switching a unit off. Dropout switches off random units during training so that units cannot depend too much on each other, which helps against overfitting (Srivastava et al., 2014).

Experimental Comparison

The second model differs from the baseline in one aspect: the dropout rate is 0.3 instead of 0.1. In my code, this one value is used inside both encoder layers and before the output layer, so these places change together.

I chose this change because of how the baseline trained. Its validation loss was lowest at epoch 8 (0.2813) and then rose to 0.2978 at epoch 10, while its training loss kept falling to 0.1617. At epoch 10, training accuracy was 94.35% and validation accuracy was 91.04%, a gap of 3.3 points. This points to mild overfitting, and dropout is a direct way to test whether more regularization narrows the gap (Srivastava et al., 2014). Srivastava et al. (2014) describe a retention probability of 0.5 to 0.8 as typical for hidden units, which is a dropout rate between 0.2 and 0.5. The baseline rate of 0.1 is below that range and 0.3 is inside it. They also note that too little dropout may not prevent overfitting.

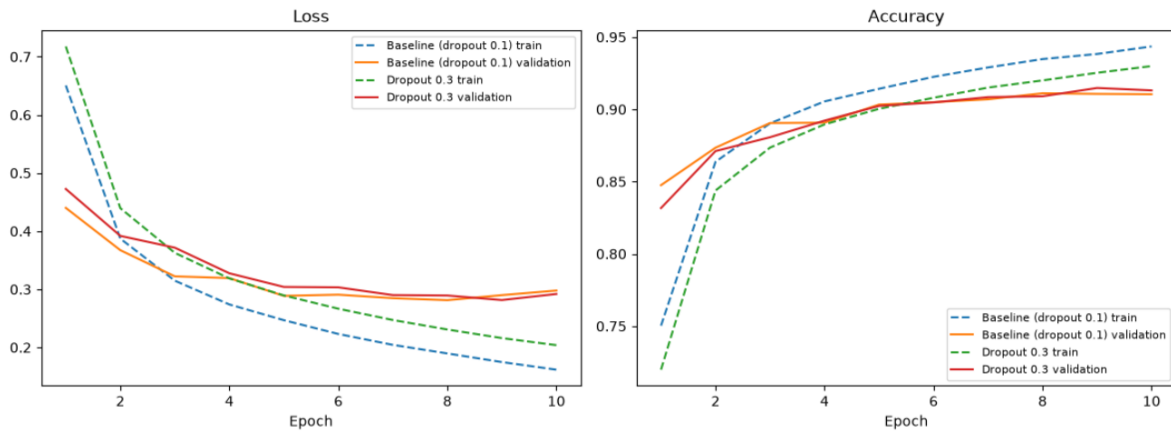
Everything else was kept the same: the dataset and splits, the vocabulary, the maximum length of 128 tokens, the architecture (2 layers, 128 dimensions, 4 heads, feed-forward size 256), the optimizer and its settings, the batch size, the 10 epochs, gradient clipping, the random seed of 42, the order of the training data, and the rule for keeping the best epoch. Dropout is random, so the two runs cannot be identical in every detail even with the same seed. Each model was trained once.

Results and Interpretation

Both models learned steadily and reached about 91% validation accuracy (Figure 1 and Table 2). The baseline fit the training data more closely: at epoch 10 the gap between training and validation accuracy was 3.3 points, against 1.7 points for dropout 0.3. The baseline's validation loss rose after epoch 8. The validation loss of the dropout 0.3 model was lowest at epoch 9 and rose only slightly at epoch 10 (0.2814 to 0.2919). Dropout 0.3 learned more slowly at first: validation accuracy after epoch 1 was 0.8317, against 0.8475 for the baseline.

Figure 1

Training and Validation Loss and Accuracy of the Baseline and the Dropout 0.3 Model

**Table 2**

Training Behavior on the Validation Set (Seed 42)

Measure	Baseline (dropout 0.1)	Dropout 0.3
Best validation accuracy	0.9111 (epoch 8)	0.9147 (epoch 9)
Lowest validation loss	0.2813 (epoch 8)	0.2814 (epoch 9)
Training accuracy at epoch 10	0.9435	0.9299
Validation accuracy at epoch 10	0.9104	0.9131
Gap at epoch 10 (training minus validation)	3.3 points	1.7 points
Validation accuracy after epoch 1	0.8475	0.8317

On the test set, the baseline reached an accuracy of 0.9058 and a macro F1 of 0.9057, and the dropout 0.3 model reached 0.9091 and 0.9088 (Table 3). The difference is 0.33 points, about 25 of the 7,600 articles. Sports is the easiest class for both models (F1 about 0.96). Business and Sci/Tech are the hardest (F1 between 0.87 and 0.89).

Table 3*Test Set Results (7,600 Articles, Evaluated Once)*

Measure	Baseline (dropout 0.1)	Dropout 0.3
Accuracy	0.9058	0.9091
Macro F1	0.9057	0.9088
F1, World	0.9063	0.9130
F1, Sports	0.9618	0.9582
F1, Business	0.8737	0.8783
F1, Sci/Tech	0.8811	0.8859

The gain from dropout 0.3 was not even across classes. Compared with the baseline, it classified 46 more World articles and 4 more Sports articles correctly, but 1 fewer Business article and 24 fewer Sci/Tech articles. The two models disagree on 381 test articles: 178 that only the baseline classified correctly and 203 that only dropout 0.3 did. An exact sign test on these counts gives $p = 0.22$, so the difference is not statistically distinguishable. Because each model was trained once, I do not claim that dropout 0.3 is better. The results do show that it narrowed the gap between training and validation accuracy.

The most common mistake in both models is confusing Business and Sci/Tech. These two classes account for 300 of the baseline's 716 errors and 288 of the 691 errors of the dropout 0.3 model (about 42% in both cases). The largest single group is Business articles predicted as Sci/Tech: 180 for the baseline and 165 for dropout 0.3, out of 1,900 Business articles. Examples from a random sample of errors are "Microsoft signs two Indian deals," "Report finds handheld device market in continued decline," and "Lucent's history a case study in wild rides." All three are business news about technology companies, so the topic sits between the two classes, and a human reader could place them in either.

In plain terms: both models sort about 9 out of 10 news articles into the right topic. Most mistakes happen on stories about the business side of technology, which fit two topics. Switching off more of the model's units while it learns (higher dropout) changed the results only slightly, and with one run per model this difference could be due to chance.

Limitations and Risks

Single runs. Each configuration was trained once with seed 42. The test accuracy difference of 0.33 points is small and not statistically significant ($p = 0.22$), so I cannot say that the dropout rate caused it.

Label quality. Some labels look questionable. In the random sample of errors, "JFK killing fades in intensity" is labeled Sci/Tech and "US football: Patriots' historic win" is labeled World, while the model predicted World and Sports, which match the content better. The dataset description I reviewed does not explain how individual labels were checked. Such cases mean some counted errors may not be real model mistakes. This is based on a sample of six errors, so it is an illustration, not a measured rate.

News-agency tags. About 20% of training articles (24,254 of 120,000) show a news-agency tag in the first 60 characters of the description, and some tags lean strongly toward one class: Canadian Press (314 articles, 100% World), NewsFactor (253 articles, 98% Sci/Tech), and washingtonpost.com (207 articles, 92% Sci/Tech). Geirhos et al. (2020) describe shortcut learning, where a model uses features that work on test data from the same source but fail under other conditions. Agency tags are a possible shortcut feature here. Tagged test articles were classified more accurately than untagged ones (0.9273 versus 0.9003 for the baseline and 0.9293 versus 0.9039 for dropout 0.3). Both models still reach about 90% without a tag, so they do not depend on tags alone. However, this comparison cannot show whether the models use the tags, because tagged articles may also be easier for other reasons. My test set comes from the same source as the training data and has a similar share of tagged articles (20.3%), so it cannot reveal reliance on tags either (Geirhos et al., 2020).

Vocabulary and length. The word-level vocabulary maps 1.30% of validation tokens to the unknown token, so new names and words are lost. Cutting texts at 128 tokens affects fewer than 0.5% of training articles, but it removes the end of the longest ones.

Scope of the data. Several sample articles describe events from 2004, for example the funeral of Yasser Arafat and an IDC report for the third quarter of 2004. The vocabulary and topics may therefore not match current news. The dataset also has only four broad topics from one news collection.

Training and evaluation. I did not search for better hyperparameters. The best epoch was chosen on the validation set, so validation scores are slightly optimistic: the test accuracy was about 0.5 points below the best validation accuracy for both models (0.9058 versus 0.9111 and 0.9091 versus 0.9147). I evaluated accuracy, precision, recall, F1, and confusion matrices, and I did not test the models on articles from other sources.

Ethical and Responsible Use

Wrong sections for readers. If a model like this sorted news automatically with no human review, about 9% of Business articles (180 and 165 of 1,900 in the two models) would be sent to Sci/Tech on this test set. In a news product, this would hide some articles from the readers who look for them. A human check, especially for articles about technology companies, would reduce this risk.

Source bias. Among the tagged articles, about 97% come from three wire services (Reuters, AP, and AFP: 23,480 of 24,254). I do not know the sources of the other 80% of the articles. The model may fit the style and coverage of the common sources better than those of rare ones. Geirhos et al. (2020) describe how dataset biases can make models work well on common cases and worse on groups that are rare in the data, including in fairness settings. I did not test this, so it is a risk and not a finding.

Responsible use. Results should be reported per class and not only as overall accuracy, because the classes differ (F1 from about 0.87 to 0.96). The topic labels should not be treated as ground truth, since some look questionable. Any use on news from other sources or time periods should be tested first.

Future Improvements

1. Test the effect of agency tags directly. Remove the tag text from the test articles and re-evaluate the trained models, or evaluate them on articles from other sources. Geirhos et al. (2020) recommend this kind of test on data that differs from the training data.
2. Repeat each configuration with several random seeds and report the mean and the range, so that small differences can be judged.
3. Compare dropout rates with the same, longer training for all models. Srivastava et al. (2014) note that stronger dropout can need more training time.
4. Try a sub-word tokenizer to reduce unknown words (1.30% of validation tokens in this work).
5. Review a larger sample of the Business and Sci/Tech errors with more than one reader, to measure how many come from ambiguous or wrong labels.

AI assistance. I used Claude (Anthropic's AI assistant) to plan this project, to help write and debug the tokenizer, model, training, and evaluation code, and to help draft this report. I ran all the code, checked the outputs, and revised the text, and all numbers come from my notebook.

References

Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., & Wichmann, F. A. (2020). Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11), 665–673.

<https://doi.org/10.1038/s42256-020-00257-z>

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15, 1929–1958.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In I. Guyon, U. von Luxburg, S. Bengio, H. M. Wallach, R. Fergus, S. V. N. Vishwanathan, & R. Garnett (Eds.), *Advances in neural information processing systems* (Vol. 30, pp. 5998–6008). Curran Associates.